Master Game Vision, Systems Blueprint, and AI Implementation Spec

Project: Simulation-first Medieval World Sandbox (v4)

Document purpose	Consolidated game vision document, systems design blueprint, and machine-readable implementation handoff for a long-term solo commercial project.
Project identity	A CDDA-like, data-driven medieval world simulator with Dwarf Fortress-inspired combat modeling and Bannerlord-style army and settlement progression.
Primary player fantasy	Simulate a living medieval world where you can do anything, starting as an ordinary person and potentially ending as a ruler or historical force.
Primary design priorities	World simulation; economic simulation; army command; emergent storytelling; brutal but playable combat realism; moddability.
Target mode	Single-player first, moddable from day one, future-friendly for async world sharing and possible co-op extensions.
Core technical doctrine	Smart abstraction at scale, deep local simulation, JSON-driven content, stable save migrations, simulation truth separated from rendering.
Status	Pre-production vision and architecture baseline for AI-assisted design, planning, and implementation.

1. Game Vision Document

1.1 High-level thesis

The project is a simulation-first medieval sandbox in which the world is the protagonist and the player is a disruptive variable inside it. The game should feel like a living feudal world that continues to evolve whether the player acts, pauses, retires, or dies. The player begins with human-scale needs such as shelter, food, work, and safety, and can scale upward into ownership, command, political leverage, and historical legacy.

1.2 Pillars

- Living world simulation: economies, settlements, trade, wars, migration, weather, politics, and fire/destruction continue off-screen through lower-fidelity simulation.
- Deep character-in-world play: detailed character creation, use-based growth, layered stats and perks, property, professions, social rank, and knowledge as progression.
- Dwarf Fortress-inspired combat: body-part injuries, armor coverage by material, weapon reach and momentum, bleeding, pain, and shock with harsh but survivable outcomes.
- Rise to command: the player can remain civilian or scale into mercenary command, settlement ownership, factional power, and eventual rulership.

- Data-driven moddability: major content and tuning are defined in external data and validated by schemas rather than hardcoded content tables.

1.3 Player arc

The player starts as an ordinary entity with no plot privilege. Early play focuses on survival, income, housing, and entry into a local profession. Mid game focuses on compounding systems such as workshops, caravans, companions, property, and local armed power. Late game focuses on armies, settlements, claims, vassalage, factional leverage, kingdom management, and long-term historical impact. Optional retirement and heir continuation allow the player to watch the world persist across decades or centuries.

1.4 Tone and setting

The setting should feel grounded and historical in its social and economic logic, but it may contain dungeon-only supernatural anomalies, rare monsters, and later-world escalation such as undead incursions. The baseline emotional tone is indifferent, harsh, and playable rather than apocalyptic. Society should feel constrained by feudal stagnation and endemic conflict, not guided by heroic quest structure.

1.5 Core player activities

- Secure food, shelter, and local income in the first hour through work, trade, military service, scavenging, or small production.
- Develop a profession such as trader, soldier, crafter, landholder, or outlaw while building assets and relationships.
- Run workshops, caravans, camps, and small properties that generate returns.
- Recruit and manage followers, hire specialists, and develop small armed forces.
- Lead formations in WEGO battles with order-driven play rather than direct action control.
- Found, improve, defend, and administrate settlements.
- Participate in faction politics through fealty, betrayal, claims, landholding, or new faction creation.
- Explore dungeons as high-risk local exceptions to the grounded simulation layer.

1.6 Non-goals for early production

- Do not treat deep religion, deep culture simulation, rich quest structure, full dynastic politics, and global undead invasions as launch blockers.
- Do not require continent-scale 1500-unit battles in the first playable build.
- Do not require fully persistent identity for every soldier at launch.
- Do not prioritize content quantity over system coherence.

2. Systems Design Blueprint

2.1 Architectural rule

The simulation must be built as layered fidelity: strategic world simulation, regional operational simulation, and tactical local simulation. Systems must degrade gracefully as fidelity drops. No major system should freeze simply because the player is not present. The engine should downgrade detail, not stop causality.

2.2 Simulation layers

- Strategic layer: full-world map, long-range time pulses, faction pressure, migration, weather regimes, trade volumes, settlement growth/decline, and political event resolution.
- Regional layer: route-level trade, settlement inventories and production chains, local crises, party movement, regional conflict pressure, and off-screen event approximation.
- Tactical layer: local tile and z-level maps, direct actors, detailed combat, local inventories, construction stages, local hazards, and direct player interaction.

```
Simulation downgrade rule:
Tactical -> Regional -> Strategic
Detailed state becomes summarized state, not deleted state.
```

2.3 Core subsystems

- World generation: hybrid handcrafted rules + procedural generation of terrain, regions, resources, settlements, routes, and optional pre-game history.
- Entity registry: stable IDs for people, settlements, parties, factions, items, buildings, events, and map cells/chunks.
- Economy engine: realistic production chains, regional specialization, price discovery, demand pressure, conceptual transport pulses, and acreage-linked yield accounting with seasonal and labor modifiers. Bulk commodities should resolve through ledgers unless the player enters local/tactical detail.
- Settlement manager: core management layer for population classes, goods, labor slots, ownership, growth, law variation, notable NPCs, and local construction.
- Character system: deep creation, attributes, traits, backgrounds, use-based skills, perks, reputation, and knowledge state.
- Combat engine: WEGO order planning, per-combatant state, body zones, wounds, armor coverage, reach, momentum, terrain, formation effects, and recovery.
- Army system: recruitment, troop templates plus selective persistent identities, training, pay, soft logistics, formations, and command structures without chain-of-command micromanagement.
- Property and ownership: camps, homesteads, forts, villages, town improvements, and staffing/manpower controls.
- Law and politics: local law variants, crimes, claims, fealty, faction relations, succession hooks, and deferred deep politics.
- Dungeon system: local procedural exception spaces with z-level traversal, loot risk, and optional supernatural content.

2.4 Technical constraints implied by design

- Rendering must not be the source of truth. Simulation state must survive absent scenes, unloaded maps, and UI changes.
- JSON-first definitions are mandatory for moddability. Runtime validation is required.
  - Runtime architecture uses a hybrid model: ECS for active entities and tactical simulation, domain ledgers/state stores for aggregate economy, history, and strategic world summaries. Do not force all world logic into ECS when ledger simulation is a better fit.
- Save files must be versioned and migratable from the beginning.
- Every expensive system must have a reduced-fidelity equivalent for off-screen execution.
- Battles are a specialized subsystem and cannot simply reuse low-fidelity world travel logic.
- The engine choice matters less than keeping the simulation core decoupled from presentation.

2.5 Vertical slice doctrine

The first useful slice is one region, not a full world. It should support a generated region with 5 to 12 settlements, 2 to 4 factions, a working economy, movement of trade parties, small-scale combat, deep character creation, limited property ownership, and a visible snowball loop. If this slice is not fun, continent scaling is irrelevant.

3. AI-readable Master Spec

3.1 Machine consumption contract

Another AI or implementation agent should treat this document as normative unless a later document explicitly supersedes it. The AI must prefer preserving system invariants over adding new features. Ambiguities should resolve toward smart abstraction, regional-first delivery, and data-driven architecture.

```
{
  "spec_version": "4.0.0",
  "project_code": "MEDIEVAL_WORLDSIM",
  "authority": "normative",
  "design_priority_order": [
    "world_simulation",
    "economic_simulation",
    "army_command",
    "emergent_storytelling",
    "combat_realism",
    "moddability"
  ],
  "non_negotiables": [
    "economic_simulation",
    "dwarf_fortress_style_combat_modeling",
    "settlement_managers"
  ],
  "mvp_doctrine": "regional_sandbox_first",
  "player_status": "ordinary_entity_in_sim"
}
```

3.2 Canonical design decisions

- The player is not narratively special.
- Permadeath, heir continuation, and severe-setback modes should exist as optional settings.
- The world target is full-world scale in the long term, but implementation must start with a deep region.
- History generation before game start is supported but optional.
- Combat is WEGO and order-driven, with the player acting primarily as commander during battles.
- Morale, panic, routing, surrender, and desertion are not core systems in the first design baseline.
- Terrain and formation matter heavily.

- Siege depth is a long-term target; early versions should not promise full Bannerlord-scale siege engineering fidelity.
- Logistics matter lightly: present, meaningful, but not punishing.
- Warfare can be ignored by the player while still leaving a deep civilian and economic game path.
  - Architecture baseline is hybrid ECS plus ledger simulation: ECS owns runtime actors, local objects, and tactical hazards; aggregate stores own production ledgers, region summaries, historical records, and other high-volume off-screen state.
  - Production must be realistic in causal structure. Agriculture, extraction, and manufacturing should be grounded in measurable input-output models, but represented at the coarsest fidelity that preserves believable economic behavior.
  - Regional settlement generation uses a discrete grid. One building footprint occupies one regional map cell unless a special structure definition explicitly spans multiple cells. This rule exists for layout clarity, placement logic, and deterministic settlement growth.

3.3 Module registry

Module	Responsibility	Inputs	Outputs
worldgen	Generate terrain, resources, regions, routes, and settlement seeds.	Seed config, handcrafted rules, content defs	World map, region graph, resource fields
history_sim	Optional pre-game history pass.	World state, faction templates	Historical events, dynasties, ruins, claims
econ_core	Run production, demand, price, and scarcity.	Settlement state, route state, goods defs	Updated inventories, prices, shortages
settlement_core	Run growth, labor, ownership, construction, and local services.	Population classes, buildings, goods, laws	Settlement updates, tasks, local pressure
party_core	Move parties and resolve route-level interactions.	Route graph, party defs, region state	Party positions, encounter hooks
combat_core	Resolve WEGO battles with per-entity injury modeling.	Orders, combatants, terrain, equipment	Wounds, deaths, equipment changes, battle outcome
property_core	Handle player and NPC ownership.	Title data, buildings, staffing, budgets	Income, upkeep, staffing state
law_politics	Track crimes, legal reaction, claims, and faction relations.	Events, laws, faction state	Legal status, relation shifts, claim pressure
save_migration	Version, validate, and migrate saves.	Old save schema, content registries	Current schema save state

All modules should communicate through explicit state objects and event queues rather than hidden scene references.

3.4 Data contracts

```
Core state objects (illustrative):

WorldState
- version
- seed
```

```
- current_day
- region_ids[]
- faction_ids[]
- global_event_queue[]

SettlementState
- settlement_id
- region_id
- population_classes{}
- goods_inventory{}
- building_slots[]
- production_jobs[]
- local_laws[]
- notable_person_ids[]
- prosperity
- unrest

PersonState
- person_id
- traits[]
- skills{}
- body_state{}
- social_links[]
- reputation{}
- ownership_refs[]
- active_role

CombatantState
- combatant_id
- team_id
- tile_pos
- z_level
- body_zones{}
- equipment_refs[]
- planned_orders[]
- resolved_actions[]

SaveSchema
- schema_version
- active_mods[]
- registry_hashes{}
- migration_history[]
```

All data definitions should be schema-validated at load time. Invalid data should fail fast in debug builds and fail gracefully with explicit logging in release builds.

3.5 AI implementation rules

1. Do not expand scope before the regional sandbox loop is playable.
2. Prefer reusable simulation infrastructure over bespoke one-off mechanics.
3. When a feature can be represented as data plus a generic rule, do not hardcode it.
4. Preserve save compatibility and stable IDs when refactoring.
5. When a system is too expensive, downgrade fidelity before deleting continuity.
6. Do not add narrative quest dependence to solve sandbox motivation problems until systemic incentives fail.
7. Treat debugging tools, state inspectors, and simulation visualizers as mandatory production features.
8. Any combat implementation that cannot scale to dozens of tactical actors in WEGO with detailed injuries is incomplete.

3.6 Validation and acceptance criteria

- A generated region can run for a simulated period without catastrophic economy collapse unless triggered by internal conditions.
- Settlements can produce, consume, import, and fail in believable ways.
- A player can secure housing and income without mandatory combat.
- A small battle can resolve with body-part injury outcomes, equipment effects, and tactical order influence.
- A player can own or manage a small productive asset that creates returns over time.
- Save files survive at least one schema iteration through migration tests.
- External content definitions can be extended by mods without code changes for at least one major content domain.

4. Technical Roadmap and Timeline

4.1 Delivery posture

This is a serious solo long-term commercial project. The roadmap must therefore optimize for compounding technical leverage rather than broad feature count. Each phase must produce reusable systems, debugging leverage, and a demonstrably playable outcome. The first public-facing commercial promise should be a regional sandbox, not the final continent-scale vision.

4.2 Phase plan

Phase	Time	Goal	Key deliverables	Exit criteria
0. Technical foundation	1-2 months	Project skeleton, registries, save versioning, debug tools.	Core project structure; data loader; schema validator; state inspector; deterministic tick loop.	Simulation clock runs; content loads from external files; saves serialize and reload.
1. World and region generation	2-3 months	Generate the first deep region.	Terrain; resources; settlement placement; route graph; optional lightweight history pass.	A region can be generated repeatedly with logical settlement placement and visible route structure.
2. Regional economy and settlement core	3-4 months	Make the world function without the player.	Goods, prices, production chains, settlement inventories, labor slots, moving trade parties.	Regional economy runs and settlements can prosper or decline.

3. Character and local map layer	2-3 months	Place the player into the world.	Character creator; local map chunks; z-level support baseline; movement; interaction; housing and basic work loops.	The player can spawn, travel locally, earn basic income, and secure shelter.
4. Combat vertical slice	3-4 months	Prove deep tactical combat.	WEGO orders; small tactical maps; body zones; wounds; armor coverage; weapons and terrain effects.	Small battles are readable, tactically influenced, and mechanically deep.
5. Snowball systems	3-4 months	Make wealth and influence compound.	Workshops; camps; small ownership; followers; basic recruitment; small armed group.	The player can create a self-reinforcing income or power loop.
6. Regional sandbox alpha	3-5 months	Unify the slice into a compelling product core.	Faction pressure; local law; more professions; management UI; stabilization pass.	The game is fun as a regional sandbox without needing the full-world promise.
7. Scale-out architecture	4-8 months	Expand toward world scale.	Multi-region streaming; broader faction logic; larger armies; abstraction upgrades; long-run sim tests.	Multiple regions run coherently and performance stays acceptable.
8. Long-tail expansions	ongoing	Add deferred systems.	Dynasties; deeper politics; religion/culture; sieges; dungeons expansion; supernatural world events.	Expansion systems integrate without breaking the core loop.

For a serious solo developer, a strong regional sandbox alpha is the realistic major target within roughly 18 to 28 months if scope discipline holds. Full-world commercial breadth will take materially longer.

4.3 Milestone gates

9. Gate A: World logic exists without the player.
10. Gate B: The player can live in the world without combat-only progression.
11. Gate C: Tactical combat is deep enough to justify its cost.
12. Gate D: A compounding snowball loop creates clear sandbox fun.
13. Gate E: The regional sandbox stands on its own as a product core.
14. Gate F: Only after Gate E should world-scale expansion meaningfully accelerate.

5. Risks, Tradeoffs, and Control Measures

5.1 Critical risks

- Scope collapse from trying to ship the continent-scale vision before the regional slice is good.
- Performance collapse from over-literal full simulation in battles or off-screen systems.
- Architecture rot from mixing engine scene state with authoritative simulation state.
- Save corruption and mod breakage if versioning and registries are deferred.
- Design drift if dungeons, undead, or narrative content overshadow the simulation core too early.

5.2 Explicit tradeoff decisions

- Choose huge long-term world scope, but implement one deep region first.
- Choose playable friction over pure brutal realism.
- Choose smart abstraction over naive individual simulation everywhere.
- Choose army-command emphasis over personal action-combat emphasis.
- Choose pure sandbox over scripted mainline progression.
- Choose persistent world history over tight authored pacing.
- Choose system complexity, but deliver it through readable interfaces and controlled simulation layers.

5.3 Change control

Any proposed feature that adds simulation depth, content load, or cross-system coupling must be evaluated against three questions: Does it strengthen the regional sandbox loop; does it preserve moddable data architecture; does it delay first fun. If the answer is no, no, yes, it should be deferred.

6. Final Design Baseline

6.1 Summary

The correct first target is not to build the entire dream. The correct first target is to build a fun regional sandbox that proves economic simulation, deep combat modeling, and settlement-oriented snowball progression. Once that works, scaling outward becomes an engineering exercise. Until then, anything beyond the slice is speculation.

Appendix A. First Build Queue

15. Implement deterministic tick loop and unified time service.
16. Implement registry system for IDs, data defs, and schema validation.
17. Implement WorldState and SettlementState with serialization.
18. Implement region generator and route graph.
19. Implement goods defs and the first production chain set.
20. Implement settlement manager pulse updates.
21. Implement party movement between settlements.
22. Implement character creation and spawn flow.
23. Implement local map with z-level baseline.
24. Implement work, shelter, and income loops.
25. Implement WEGO tactical prototype with 4 to 12 combatants.
26. Implement body-zone injuries, armor coverage, and recovery.

Appendix A1. Runtime Architecture Addendum (v4)

A1.1 Hybrid ECS plus domain-ledger architecture

The implementation baseline is hybrid, not pure. ECS should handle runtime entities that benefit from composable components and frequent updates. Domain ledgers and summary stores should handle high-volume aggregate state where accounting, historical continuity, and strategic pulses matter more than per-entity behavior.

- Use ECS for: characters, combatants, companions, animals, moving parties, local map props, projectiles, fires, construction jobs in loaded areas, and other tactical or near-tactical entities.
- Use domain ledgers/state stores for: settlement production, acreage assignment, reserve stocks, regional trade balances, historical events, migration summaries, macro weather state, and other aggregate off-screen systems.
- Entity IDs must remain stable across ECS and non-ECS domains. ECS instances may be created, collapsed, or rehydrated, but the authoritative identity record must survive.
- Systems should communicate through explicit events and state deltas. Rendering, scenes, and UI are observers and controllers, not sources of truth.
- Do not attempt a pure ECS world where every acre, peasant, and strategic summary is forced into components if a ledger model is more maintainable and faster.

A1.2 ECS component baseline

Illustrative runtime component set for tactical and active regional entities:

- IdentityComponent: stable_id, template_id, name_ref, faction_id, culture_id.
- TransformComponent: region_cell, local_tile, z_level, facing, movement_state.
- BodyComponent: body zones, hit tables, wound slots, pain, bleeding, shock thresholds.
- EquipmentComponent: equipped item refs, armor coverage refs, loadout summary.
- IntentComponent: current order package, queued actions, tactical stance, formation slot.
- NeedsComponent: hunger, sleep debt, temperature stress, morale-lite state.
- EconomyActorComponent: wages, carry capacity, trade task link, work assignment link.
- LifecycleComponent: age, health trend, collapse eligibility, persistence priority.
- CombatStatsComponent: skill refs, reach, momentum class, attack cadence, defense values.
- HazardComponent (for fires, collapses, etc.): intensity, spread class, damage pulse.

A1.3 System scheduling rule

Process systems in deterministic phases: input and orders, world pulse, production pulse, movement, hazard resolution, combat resolution, persistence collapse/rehydration, then presentation sync. Tactical systems may tick at a finer cadence than strategic pulses, but the schedule must remain deterministic for saves, replays, and debugging.

Appendix A2. Realistic Production and Acreage Model

A2.1 Production doctrine

Production must be realistic in causal logic. That means outputs are constrained by land, labor, tools, season, inputs, and disruption. It does not mean every sack, bushel, or ingot must exist as an

always-simulated item object. Bulk production should use ledgers until the player enters a local context where discrete objects matter.

- Agriculture uses acreage and yield units. Example canonical unit pair: acres and bushels.
- Extraction uses site capacity and labor throughput. Example canonical unit pair: shafts or cuts and ore tons.
- Manufacturing uses input batches and worker-days. Example canonical unit pair: recipe batch and finished goods count.
- Transport may use conceptual pulses at strategic scale and discrete cargo only in loaded local contexts.
- Every production domain must expose both expected output and failure conditions so famine, shortage, and economic collapse can emerge from the same rules.

A2.2 Acreage-linked agriculture baseline

Each regional world cell must define a land budget. The baseline assumption may be 250 acres per regional cell, but this should remain data-driven so future world scales can adjust without code changes. The number only makes sense if regional cell scale is defined consistently across world generation, settlement footprints, and route distances.

- total_acres = full land represented by the regional cell.
- arable_acres = portion suitable for crops.
- worked_acres = acres actively farmed this season.
- fallow_acres = reserved recovery land.
- pasture_acres = grazing land.
- woodlot_acres = managed timber/fuel land.

Baseline agriculture formula: seasonal_output = worked_acres x base_yield_per_acre x fertility_modifier x weather_modifier x labor_modifier x tool_modifier x disruption_modifier. Seed retention, spoilage, tax extraction, household consumption, reserve storage, and market surplus must all derive from that output before tradeable surplus is emitted.

This model should produce realistic consequences: raids reduce labor and tool availability, drought reduces weather modifier, over-expansion can cut fallow recovery, and settlement growth increases pressure to convert new acres or import food.

A2.3 Realism boundary

Realistic does not mean equally microscopic everywhere. The requirement is realistic constraint and believable balances. Use the same accounting rigor across food, timber, ore, livestock, cloth, tools, and weapons so agriculture does not become over-detailed while the rest of the economy remains fake.

Appendix A3. Regional Settlement Grid and Footprint Rule

A3.1 Settlement generation grid

Settlement generation should use a discrete regional settlement grid layered inside each settlement-bearing regional cell. The baseline rule is that one ordinary building occupies one settlement-grid unit. This is similar in spirit to CDDA city generation in that coarse layout is decided on a higher-order grid before local map detail is instantiated.

- One building definition maps to one regional settlement-grid cell by default.
- Large structures may explicitly declare multi-cell footprints, but this must be opt-in by template.
- Roads, walls, plazas, and fields may occupy dedicated non-building cells.
- Cell placement rules should consider road adjacency, district tags, terrain suitability, ownership, and service radius.
- Local tactical maps are generated from the selected settlement-grid cell and its template, not stored as always-loaded detail.

A3.2 Why the one-cell rule makes sense

Yes, this makes sense, provided the regional settlement grid is understood as a generation and management abstraction, not a literal one-building-equals-one-tactical-room model. The one-cell rule keeps settlement generation readable, deterministic, and expandable. It also makes zoning, district growth, service coverage, and path planning tractable for a solo-developed simulation.

When the player enters a building, that single regional cell can expand into a richer local layout with multiple rooms, courtyards, vertical stories, attached yards, or dungeon entrances. One cell is the strategic footprint, not the full local blueprint.

Appendix B. Immediate engine guidance

Godot remains the most practical default for a solo developer because it supports fast 2D iteration, tooling flexibility, custom editors, and open data workflows. The real rule is that the simulation core should be engine-agnostic enough that rendering, UI, and scene management can change without invalidating the authoritative world state.